

Módulo 9

Auto-Vectorização

Copie o ficheiro `/share/acomp/LOOP-P09.zip` para a sua directoria, faça o unzip e gere os executáveis com nível de optimização O2 e 2 opções diferentes: `novect-kernel` não inclui vectorização, enquanto `vec-kernel` inclui a opção de auto-vectorização pelo compilador. Para esse efeito, basta escrever `./mymake.sh`. Este comando gerará também os ficheiros `vec-kernel.s` e `novect-kernel.s` contendo o *assembly* correspondente aos diferentes níveis de optimização.

Ao longo deste módulo iremos concentrar a nossa atenção nas funções `loop1()` a `loop6()`, `loop_AoS()` e `loop_SoA()` que se encontram em `vec-kernel.c`.

Exercício 1 - Para cada uma das funções `loop1()` a `loop6()` e `loop_AoS()` examine o respectivo código e indique justificando, na coluna “Obs.” da Tabela 1, se pensa tratar-se, ou não, de código vectorizável pelo compilador. As linhas correspondentes a `loop3-V2()` e `loop_SoA()` serão preenchidas nos exercícios posteriores.

Preencha a tabela para ambas as versões, cuja principal diferença será a vectorização. Note que pode verificar a vectorização consultando os ficheiros `.s`. Para executar cada versão da função `loop()` use o comando `sbatch` conforme indicado abaixo para a versão `loop1()`. Este comando executará ambos os executáveis, isto é, `novect-kernel` e `vec-kernel`. O argumento usado para seleccionar a versão pode ser consultado na coluna `V` da Tabela 1:

```
sbatch loop.sh 1
```

Exercício 2 - Analize a função `loop3()`. Porque é que não vectoriza? Consegue sugerir uma alteração ao código desta função de forma que vectorize? Se sim, altere-a e preencha a linha `loop3-V2()` da tabela.

Exercício 3 - Escreva a função `loop_SoA()`, para que realize os mesmos cálculos que `loop_AoS()` mas usando uma estrutura de *arrays*.

Note que:

- o tipo de dados `My_SoA` já está definido em `vec-kernel.h`:
`typedef struct {float *a, *c} MY_SoA;`
- a variável `SoA` é declarada em `main.h` e os respectivos vectores (`a` e `c`) devidamente inicializados.

Exercício Complementar

Considere a função `loop5()`, que não vectoriza devido a uma dependência RAW. Consegue reorganizar as computações de forma que estas vectorizem parcialmente?

Sugestão: Pode separar o ciclo em dois ciclos, um deles sem dependências!

func	v	opt	T _{exec} (ms)	#I (M)	CPI	#VEC_SP (M)	Obs .
loop1 ()	1	no	237	550.001	1.3	0.0	vetoriza
		vec	62	81.250	2.3	350.217	
loop2 ()	2	no	123	275	1.3	0.0	não vetoriza, stride
		vec	123	275	1.3	0.0	
loop3 ()	3	no	229	650.001	1.1	0.0	não vetoriza, têm um if
		vec	229	650.001	1.1	0.0	
loop3-v2 ()	3	no	237	750.001	1.1	0.0	vetoriza
		vec	63	93.75	2.0	400.296	
loop4 ()	4	no	4137	15322.91	0.8	0.0	não vetoriza, função
		vec	4137	15322.91	0.8	0.0	
loop5 ()	5	no	243	600.01	1.3	0.0	não vetoriza, dependência de dados i-1
		vec	243	600.01	1.3	0.0	
loop6 ()	6	no	230	600.01	1.2	0.0	vetoriza
		vec	62	100.0	1.9	400.288	
loop_AoS ()	7	no	236	550.001	1.3	0.0	não vetoriza
		vec	236	550.001	1.3	0.0	
loop_SoA ()	8	no	237	550.01	1.3	0.0	vetoriza
		vec	62	81.25	2.3	350.225	

Tabela 1